Ramzor Bridge — Architecture & Connectivity Decision

ESP32 Bridge #2 companion system for the Ramzor traffic-light / train-control toys | Working document

1. Goal

The Ramzor traffic light and the train_ctrl_c3 controller are standalone ESP-NOW devices. Bridge #2 (a classic ESP32 dev board) listens for their ESP-NOW messages, resolves the sender's MAC to a friendly device name, and reports each event as JSON. The original setup piped that JSON to a Windows laptop over USB serial. The goal of this phase is to make that reporting link more robust and to allow more than one listener (a Windows app now, an Android app later) to receive the same events — ideally without a fragile physical cable.

2. System Context

- **Wemos (ESP8266)** — Ramzor traffic light. Sends state changes via ESP-NOW, unicast to Bridge #2's MAC.
- **train_ctrl_c3 (ESP32-C3)** — motor/LED test controller. Sends a boot announcement via ESP-NOW to Bridge #2. Pure ESP-NOW, no WiFi association.
- **Bridge #2 (ESP32 dev board, MAC CC:DB:A7:69:97:DC)** — receives ESP-NOW from all sender devices, builds JSON, and forwards it onward. This is the piece under redesign.
- **Consumer(s)** — a Windows app (existing, Python) today; an Android app planned for later. Both need the same event stream.

3. Options Considered

3.1 Physical connectivity from Bridge #2 to a consumer

Option	Summary	Outcome
USB serial → Raspberry Pi 4 + touchscreen	Bridge plugs into a Pi running a Python listener and a kiosk display, so the whole setup is standalone (no laptop).	Abandoned — display timing (HDMI/KMS vs FKMS), SD-card corruption from an unsafe Windows "repair", and general setup time made this too costly for the value gained.
USB serial → Windows PC directly	Plug the bridge into the PC running `bridge_listener.py`, as originally built.	Works, but ties the bridge to one physical machine/port and re-introduces cable/driver/COM-port fragility (already hit: bad cable, port renumbering).
MQTT over WiFi (bridge joins the home router)	Bridge joins WiFi, publishes JSON to a Mosquitto broker running on the PC. Any number of subscribers (PC, Android) can listen.	Broker + listener path fully proven working. **However:** joining the router forces the bridge's WiFi radio onto whatever channel the router assigns (channel 8, observed), and ESP-NOW requires sender and receiver to share a channel. This broke reception from train_ctrl_c3 and the Wemos, which stay on the ESP-NOW default channel and have no router awareness. No admin access to the router to pin its channel.
Two ESP32 split (one pure ESP-NOW → UART → one pure WiFi/MQTT)	Separate the two radio roles onto two boards wired by UART, so neither radio's requirements constrain the other.	Technically sound and channel-conflict-free, but adds a second board, extra wiring, and a second firmware project to maintain.
Bluetooth Classic SPP (bridge stays pure ESP-NOW)	**Bridge reverts to pure ESP-NOW (no WiFi, no router dependency, no channel conflict). Adds a Bluetooth Classic serial profile; the PC pairs once and gets a stable virtual COM port — no physical cable, no USB enumeration issues, no channel sharing with ESP-NOW.**	**Selected. Confirmed Bridge #2's chip (classic ESP32, not the C3 used for train_ctrl_c3) supports Bluetooth Classic/SPP. Keeps every sender device's firmware completely untouched.**

3.2 Getting from the PC to multiple consumers (incl. future Android app)

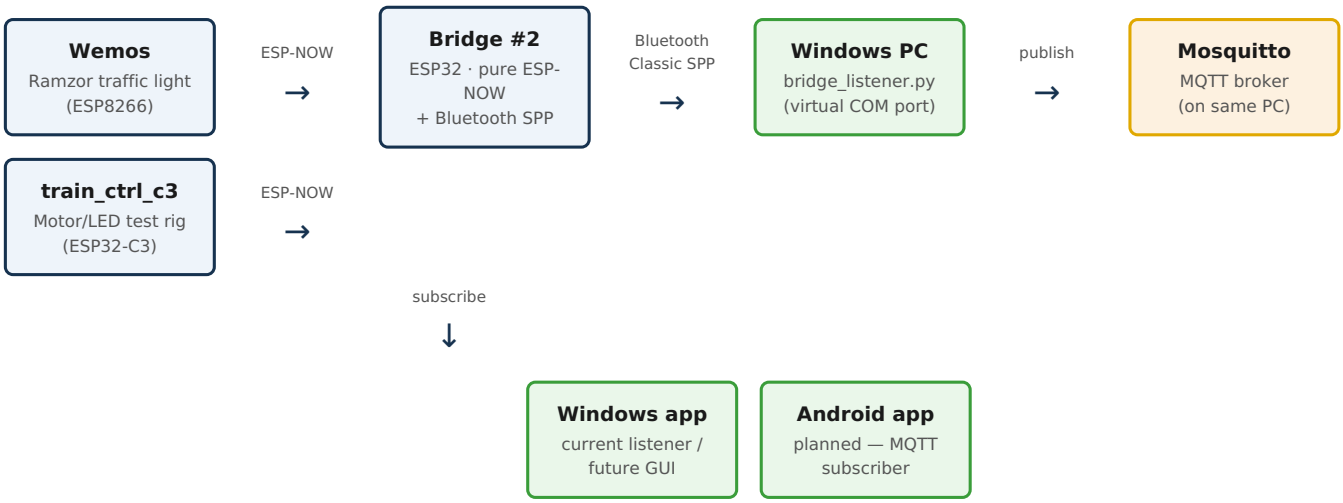
Independent of how the bridge reaches the PC, MQTT remains the right layer for fan-out to multiple consumers: the Mosquitto broker set up on the PC during this phase is retained, with `bridge_listener.py` (now fed by Bluetooth instead of WiFi) republishing each message onward. An Android app can subscribe to the same broker/topic directly (e.g. via the Eclipse Paho MQTT Android client), no different from any other subscriber.

DECISION
Bridge #2 will revert to pure ESP-NOW (no WiFi, no router dependency) and add Bluetooth Classic SPP as its serial output. The Windows PC pairs with the bridge over Bluetooth, and `bridge_listener.py` reads from the resulting stable virtual COM port and republishes each message to the existing Mosquitto MQTT broker for any number of downstream consumers (current Python tooling now, an Android app later).

Note: this reverts the WiFi/MQTT-on-ESP32 firmware changes made earlier in this phase, and undoes the router-channel-matching workaround added to train_ctrl_c3 — that workaround is no longer needed once the bridge stops joining WiFi.

4. Architecture Diagram

Planned end-state data flow



5. Next Steps (high level)

- Revert Bridge #2 firmware to pure ESP-NOW (remove WiFi connect / MQTT publish code added this phase).
- Revert train_ctrl_c3's router-channel-scan workaround — no longer needed once the bridge stops joining WiFi.
- Add Bluetooth Classic SPP output to Bridge #2 firmware, publishing the same JSON line format already in use.
- Pair the ESP32 with the Windows PC over Bluetooth and confirm a stable virtual COM port.
- Update `bridge_listener.py` to read from the Bluetooth COM port and republish each message to the existing Mosquitto broker.
- End-to-end test: power up Wemos and train_ctrl_c3, confirm events reach the MQTT broker via Bluetooth.
- Prototype an Android MQTT subscriber (generic app first, e.g. IoT MQTT Panel, to validate; custom Kotlin app later).